Communication Theory **2026**

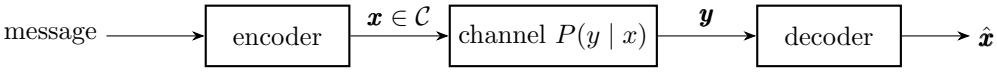
Lecture 1: Channels and maximum-likelihood decoding

Lecturer: S. Yang

Disclaimer: *These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.*

In the demo, a phone recovered a message from a noisy recording. The microphone does not hear bits, it hears numbers. What should the receiver do with them? This lecture answers the question in three steps: what a channel is, what the receiver must keep from what it observed, and what the best possible decoder does with it. A channel code is designed for a channel, and its guarantee only makes sense for that channel, so the channel comes first. Codes come next lecture.

1.1 The setting: channel, code, decoder



A **channel** has an input alphabet $\mathcal{X}$, an output alphabet $\mathcal{Y}$ and a transition law $P(y \mid x)$: the probability of observing y when x is sent (P is a probability for discrete y and a density for continuous y). Over n uses, the channel is **memoryless** if

$$P(\boldsymbol{y} \mid \boldsymbol{x}) = \prod_{i=1}^{n} P(y_i \mid x_i), \qquad \boldsymbol{x} = (x_1, \ldots, x_n),\ \boldsymbol{y} = (y_1, \ldots, y_n). \tag{1.1}$$

A **code** of length n is a set $\mathcal{C} \subseteq \mathcal{X}^n$ of M **codewords**. The encoder maps each of the M possible messages to one codeword. The **rate** of the code is

$$R := \frac{\log_2 M}{n} \quad \text{bits per channel use.}$$

A binary code that carries k information bits has $M = 2^k$ codewords and rate $R = k/n$.

A **decoder** is a map $\boldsymbol{y} \mapsto \hat{\boldsymbol{x}}(\boldsymbol{y}) \in \mathcal{C}$. It makes an error when $\hat{\boldsymbol{x}}(\boldsymbol{y}) \neq \boldsymbol{x}$, and its error probability is $P_e := P\big(\hat{\boldsymbol{x}}(\boldsymbol{y}) \neq \boldsymbol{x}\big)$. Throughout the course the messages are equally likely, so the transmitted codeword is uniform on $\mathcal{C}$.

Proposition (the best decoder). When the codeword is uniform on $\mathcal{C}$, the error probability is minimised by the **maximum-likelihood** (ML) decoder

$$\hat{\boldsymbol{x}}(\boldsymbol{y}) = \arg\max_{\boldsymbol{x} \in \mathcal{C}} P(\boldsymbol{y} \mid \boldsymbol{x}). \tag{1.2}$$

© 2026 Sheng Yang. All rights reserved.

Proof. For any decoder, the probability of a correct decision is

$$1 - P_e = \sum_{\boldsymbol{y}} P\big(\boldsymbol{x} = \hat{\boldsymbol{x}}(\boldsymbol{y}),\ \boldsymbol{y}\big) = \frac{1}{M} \sum_{\boldsymbol{y}} P\big(\boldsymbol{y} \mid \hat{\boldsymbol{x}}(\boldsymbol{y})\big) \le \frac{1}{M} \sum_{\boldsymbol{y}} \max_{\boldsymbol{x} \in \mathcal{C}} P(\boldsymbol{y} \mid \boldsymbol{x}),$$

with equality for the ML decoder (the sums are integrals for continuous $\boldsymbol{y}$). □

If several codewords reach the maximum, any of them can be chosen. For a memoryless channel, taking the logarithm of (1.1) turns the product into a sum:

$$\hat{\boldsymbol{x}} = \arg\max_{\boldsymbol{x} \in \mathcal{C}} \sum_{i=1}^{n} \ln P(y_i \mid x_i). \tag{1.3}$$

What the receiver should keep. In (1.3), the observation y_i enters only through the **likelihood function** $a \mapsto P(y_i \mid a)$, $a \in \mathcal{X}$. Multiplying this function by a positive factor that depends only on y_i shifts the scores of all codewords by the same amount, so the decoder needs it only up to such a factor.

So a good receiver hands the decoder, for each channel use, $|\mathcal{X}|$ numbers defined up to a common factor. From now until Lecture 3 the input is binary. Larger alphabets come back with Reed–Solomon codes in Lecture 4.

1.2 Binary input: one number per bit

Let the input be a bit $c \in \{0, 1\}$. We shall often write it in **antipodal form** $x = 1 - 2c$, that is, $0 \mapsto +1$ and $1 \mapsto -1$. The likelihood function has two values, $P(y \mid 0)$ and $P(y \mid 1)$. Up to a factor, it is one number, their ratio.

Definition. The **log-likelihood ratio** (LLR) of an observation y is

$$L(y) := \ln \frac{P(y \mid c = 0)}{P(y \mid c = 1)} \ \in [-\infty, +\infty]. \tag{1.4}$$

The sign of L says which bit is more likely, $L > 0$ for 0 and $L < 0$ for 1, and $|L|$ says how sure we are. If the bit is uniform, Bayes' rule gives

$$P(c = 0 \mid y) = \frac{P(y \mid 0)}{P(y \mid 0) + P(y \mid 1)} = \frac{1}{1 + e^{-L(y)}}.$$

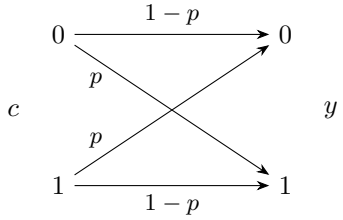
For instance, $L = 0$ means "no idea" $(1/2)$, $L = 1$ gives 0.73, $L = 3$ gives 0.95 and $L = +\infty$ means "0 for sure".

Write $P_0 = P(y \mid 0)$ and $P_1 = P(y \mid 1)$. Since $P_0/P_1 = e^L$, we have $P_0 = \sqrt{P_0 P_1}\, e^{L/2}$ and $P_1 = \sqrt{P_0 P_1}\, e^{-L/2}$. In one formula,

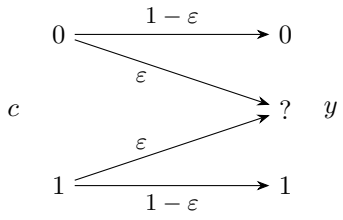
$$P(y \mid c) = \underbrace{\sqrt{P(y \mid 0)\, P(y \mid 1)}}_{\text{does not depend on } c}\; e^{\,xL(y)/2}, \qquad x = 1 - 2c. \tag{1.5}$$

The first factor is the same for both values of the bit. Everything that distinguishes 0 from 1 is in the LLR.

1.3 Three binary channels



BSC(p)



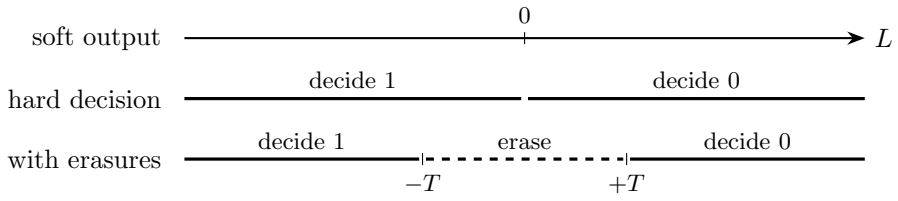
BEC(ε)

Three channels, three kinds of LLR.

- **Binary symmetric channel** BSC(p): $y \in \{0, 1\}$, and each bit is flipped with probability p. Then $L(0) = +\ell_p$ and $L(1) = -\ell_p$ with $\ell_p := \ln \frac{1-p}{p}$. Every bit comes with the same confidence.
- **Binary erasure channel** BEC(ε): $y \in \{0, 1, ?\}$, and each bit is erased with probability ε, otherwise it is received correctly. Then $L(0) = +\infty$, $L(1) = -\infty$ and $L(?) = \ln \frac{\varepsilon}{\varepsilon} = 0$. A bit is either known or completely unknown.
- **Soft-output channel**: the receiver delivers the LLR itself, $y_i = L_i$, which can be any real number. Each bit comes with its own confidence.

For example, $p = 0.1$ gives $\ell_p = \ln 9 \approx 2.2$.

The three channels can describe the same physical link, seen through three receivers. Suppose the receiver has computed an LLR for each bit.



A receiver that keeps only the sign of L makes a **hard decision** on each bit. Some decisions are wrong, and the decoder sees flipped bits: this is the BSC. A more careful receiver refuses to decide when $|L|$ is below a threshold T and declares an erasure. The BEC is the idealised version of this receiver, in which the bits that are kept are all correct. A receiver that passes L on keeps everything.

Defining the soft-output channel by its LLRs needs no assumption on the noise. In Lectures 7 and 8 we shall see how a real receiver computes them. For example, if $y = \sqrt{E_s}\,x + z$ with Gaussian noise $z \sim \mathcal{N}(0, N_0/2)$, then $L(y) = 4\sqrt{E_s}\,y/N_0$.

Remark 1.1 (every channel is a soft-output channel)**.** Take any binary-input channel and let the receiver output $\ell = L(y)$ instead of y. By (1.5), the decoder loses nothing. And the new output is its own LLR: for a discrete output,

$$P(L = \ell \mid c = 0) = \sum_{y:\, L(y)=\ell} P(y \mid 0) = \sum_{y:\, L(y)=\ell} e^{\ell}\, P(y \mid 1) = e^{\ell}\, P(L = \ell \mid c = 1).$$

So "the channel outputs LLRs" is a consistent definition, and it covers every binary-input memoryless channel. The BSC and the BEC are the special cases in which L takes two and three values.

1.4 Maximum-likelihood decoding with LLRs

A binary code is a set $\mathcal{C} \subseteq \{0,1\}^n$. For a codeword $\boldsymbol{c} = (c_1, \dots, c_n)$ we write $x_i = 1 - 2c_i$, and $L_i = L(y_i)$ is the LLR of the i-th observation.

Proposition (ML with LLRs). For a binary code on a memoryless channel,

$$\hat{\boldsymbol{c}} = \arg\max_{\boldsymbol{c} \in \mathcal{C}} \sum_{i=1}^{n} x_i L_i, \qquad x_i = 1 - 2c_i. \tag{1.6}$$

Proof. Put (1.5) in (1.3):

$$\sum_{i=1}^{n} \ln P(y_i \mid c_i) = \sum_{i=1}^{n} \ln \sqrt{P(y_i \mid 0) P(y_i \mid 1)} + \frac{1}{2} \sum_{i=1}^{n} x_i L_i,$$

and the first sum does not depend on $\boldsymbol{c}$. □

The score $\sum_i x_i L_i$ is a correlation between the codeword in antipodal form and the LLRs: a codeword gains $|L_i|$ at each position where it agrees with the sign of L_i and loses $|L_i|$ where it disagrees. Let us read (1.6) on the three channels.

BSC. Each term is $+\ell_p$ where $c_i = y_i$ and $-\ell_p$ where $c_i \neq y_i$. Let $d_H(\boldsymbol{y}, \boldsymbol{c})$ be the **Hamming distance** between $\boldsymbol{y}$ and $\boldsymbol{c}$, that is, the number of positions where they differ. Then

$$\sum_{i=1}^{n} x_i L_i = \ell_p \big(n - 2\, d_H(\boldsymbol{y}, \boldsymbol{c})\big).$$

If $p < \frac{1}{2}$, then $\ell_p > 0$ and ML is **minimum Hamming distance decoding**: pick the codeword nearest to $\boldsymbol{y}$. If $p > \frac{1}{2}$, then $\ell_p < 0$ and ML picks the *farthest* codeword. This is not a paradox: flip all received bits and the channel is a BSC$(1-p)$.

BEC. A codeword that disagrees with an unerased bit gets a term $-\infty$: it is impossible. All codewords that agree with every unerased bit have the same score. So ML is: *keep the codewords that are consistent with what was received.* If exactly one remains, it is the right one. If several remain, the decoder can only guess among them.

Soft output. Every codeword in antipodal form has $\|\boldsymbol{x}\|^2 = n$. Hence, for any $\alpha > 0$,

$$\|\boldsymbol{L} - \alpha\boldsymbol{x}\|^2 = \|\boldsymbol{L}\|^2 - 2\alpha \sum_{i=1}^{n} x_i L_i + \alpha^2 n,$$

and maximising the correlation is the same as minimising the distance. ML is **minimum Euclidean distance decoding**: pick the antipodal codeword nearest to the vector of LLRs $\boldsymbol{L} = (L_1, \ldots, L_n)$, up to scale.

Example 1.2 (one reception, three receivers)**.** Consider the code

$$\mathcal{C} = \{00000,\ 01011,\ 10101,\ 11110\},$$

with $n = 5$ and $M = 4$, so $R = 2/5$. The codeword 00000 is sent and the receiver computes the LLRs

$$\boldsymbol{L} = (+4,\ -1,\ +3,\ -1,\ +6).$$

Two LLRs have the wrong sign, but both are small. Three receivers look at them. The first passes $\boldsymbol{L}$ on. The second makes hard decisions and passes $\boldsymbol{y} = 01010$ on. The third erases the bits with $|L_i| \le 1$ and passes $0\,?\,0\,?\,0$ on.

codeword $\boldsymbol{c}$	soft: $\sum_i x_i L_i$	hard: $d_H(\boldsymbol{y}, \boldsymbol{c})$	erasures: consistent?
00000	$+\mathbf{11}$	2	**yes**
01011	$+3$	**1**	no
10101	-15	5	no
11110	$+1$	2	no

The soft decoder finds 00000, and so does the decoder with erasures. The hard-decision decoder sees $\boldsymbol{y} = 01010$, which is nearer to 01011: it picks the wrong codeword. Each decoder is the best one for what it was given. What differs is what the receiver kept.

1.5 A first code: repetition

The $(n, 1)$ **repetition code** has two codewords, $00\cdots0$ and $11\cdots1$, and rate $R = 1/n$. In antipodal form they are $(+1, \ldots, +1)$ and $(-1, \ldots, -1)$, so by (1.6) ML decides 0 if $\sum_i L_i > 0$ and 1 if $\sum_i L_i < 0$:

- BSC: majority vote;
- BEC: copy any unerased bit; guess only if all n bits are erased, which has probability ε^n;

- soft output: add the LLRs and look at the sign.

For $n = 3$ on the BSC, the decoder fails when 2 or 3 bits are flipped:

$$P_e = 3p^2(1-p) + p^3 = 3p^2 - 2p^3.$$

With $p = 0.1$, this is 0.028 instead of 0.1, and the price is a rate of $1/3$. Longer repetition codes drive P_e to zero, but the rate $1/n$ goes to zero as well.

Is this trade unavoidable? No, and this is the promise of the introduction. The BSC(0.1) has capacity $C \approx 0.53$ bit per channel use: there are codes of rate $1/2$, much longer and much smarter than repetition, whose error probability is as small as we wish. We take this as a fact until Lecture 11. The next lectures build such codes and their decoders.

1.6 What a code guarantees: the minimum distance

Definition. The **minimum distance** of a code $\mathcal{C}$ is

$$d := \min_{\boldsymbol{c} \neq \boldsymbol{c}' \in \mathcal{C}} d_H(\boldsymbol{c}, \boldsymbol{c}').$$

The repetition code has $d = n$. The code of Example 1.2 has $d = 3$.

Theorem (guarantees from d).

1. On the BSC, minimum distance decoding corrects every pattern of up to $\lfloor (d-1)/2 \rfloor$ errors.
2. On the BEC, ML decoding corrects every pattern of up to $d - 1$ erasures.

Proof. 1. Let $\boldsymbol{c}$ be sent and let $\boldsymbol{y}$ differ from $\boldsymbol{c}$ in $t \leq (d-1)/2$ positions. The Hamming distance satisfies the triangle inequality. So for any other codeword $\boldsymbol{c}'$,

$$d_H(\boldsymbol{y}, \boldsymbol{c}') \;\geq\; d_H(\boldsymbol{c}, \boldsymbol{c}') - d_H(\boldsymbol{c}, \boldsymbol{y}) \;\geq\; d - t \;>\; t = d_H(\boldsymbol{y}, \boldsymbol{c}),$$

because $2t \leq d - 1 < d$. The sent codeword is strictly the nearest.

2. Suppose at most $d - 1$ bits are erased and two different codewords are consistent with the unerased bits. They can differ only in erased positions, hence in at most $d-1$ positions. This contradicts the definition of d. So exactly one codeword is consistent, the one that was sent. □

Both guarantees are the best possible. Take two codewords $\boldsymbol{c}, \boldsymbol{c}'$ at distance d. If $\boldsymbol{c}$ is sent and $\lfloor (d-1)/2 \rfloor + 1$ of the positions where they differ are flipped, $\boldsymbol{y}$ is at least as near to $\boldsymbol{c}'$ as to $\boldsymbol{c}$. If the d positions where they differ are erased, both are consistent.

On the BEC a code corrects about twice as many erasures as it corrects errors on the BSC: knowing *where* the channel hit is worth a lot. For the code of Example 1.2, $d = 3$ guarantees

one error or two erasures. In the example two hard decisions were wrong, so the hard decoder had no guarantee, while the two erasures were within reach.

On a soft-output channel there is no such deterministic guarantee, because any number of LLRs can point the wrong way. Lecture 2 gives a probabilistic guarantee with the same d, and asks what distance costs in rate.

To remember.

- With equally likely codewords, the best decoder is ML: $\hat{\boldsymbol{x}} = \arg\max_{\boldsymbol{x}\in\mathcal{C}} P(\boldsymbol{y} \mid \boldsymbol{x})$. On a memoryless channel it needs, for each channel use, the likelihood function $a \mapsto P(y_i \mid a)$ up to a factor.
- Binary input: one number per bit, the LLR $L = \ln \frac{P(y|0)}{P(y|1)}$. BSC: $L = \pm \ln \frac{1-p}{p}$. BEC: $L \in \{+\infty, 0, -\infty\}$. Soft output: any real L.
- ML $= \arg\max_{\boldsymbol{c}\in\mathcal{C}} \sum_i x_i L_i$ with $x_i = 1 - 2c_i$: minimum Hamming distance on the BSC, consistency on the BEC, nearest antipodal codeword to $\boldsymbol{L}$ on a soft-output channel.
- The minimum distance d guarantees $\lfloor (d-1)/2 \rfloor$ errors on the BSC and $d-1$ erasures on the BEC.

Exercises

Exercise 1.1 (an asymmetric channel)**.** A binary-input channel has output $y \in \{0, 1\}$ with $P(y = 1 \mid c = 0) = 0$ and $P(y = 0 \mid c = 1) = q$, where $0 < q < 1$: a 1 is sometimes read as a 0, a 0 is never read as a 1. Compute $L(0)$ and $L(1)$. Describe the ML decoder of the $(n, 1)$ repetition code on this channel, and compute its error probability.

Exercise 1.2 (repetition, $n = 5$)**.** Compute the error probability of the $(5, 1)$ repetition code with ML decoding on the BSC(0.1). Compare with $n = 3$. What is the probability that the decoder has to guess on the BEC(0.1)?

Exercise 1.3 (hard against soft)**.** The $(3, 1)$ repetition code is used and the receiver computes $\boldsymbol{L} = (+3, -1, -1)$. What does the majority vote decide? What does ML decide? Compute $P(c = 0 \mid \boldsymbol{y})$ for a uniform bit. (Hint: by (1.5), the LLR of the whole observation is $\sum_i L_i$.)

Exercise 1.4 (three receivers again)**.** With the code of Example 1.2, the receiver computes $\boldsymbol{L} = (-3,\ +2,\ +1,\ -1,\ -5)$. Decode with the soft decoder, with hard decisions, and with erasures where $|L_i| \le 1$.

Exercise 1.5 (detection)**.** Show that a code of minimum distance d **detects** every pattern of 1 to $d-1$ errors on the BSC: the received word is not a codeword. Why can a pattern of d errors go unnoticed?